CIF Microarchitecture Specification

Supplementary Documentation

CIF Design Team

June 2026

Contents

1 Interface Signal Table **2**
- 1.1 AXI Slave Interface (AXI Master → CIF) . . . 2
- 1.2 Write Path: AW Metadata FIFO → Request Validator . . . 4
- 1.3 Write Path: Request Validator → Tag Allocator / Error Handler . . . 4
- 1.4 Write Path: Error Handler → Tag Allocator . . . 5
- 1.5 Shared: Tag Allocator → Burst Splitter . . . 5
- 1.6 Shared: Tag Allocator → Response Tracker / RTT . . . 5
- 1.7 Shared: Burst Splitter → Packet Formation Unit (Write) . . . 6
- 1.8 Write Path: W Data FIFO → Packet Formation Unit . . . 7
- 1.9 Write Path: Packet Formation Unit → Write Packet FIFO . . . 7
- 1.10 Write Path: Write Packet FIFO → MC Core . . . 7
- 1.11 Write Path: Response Tracker → B Response Generator . . . 8
- 1.12 Write Path: Response Tracker → Tag Allocator (Free) . . . 8
- 1.13 Read Path: AR Metadata FIFO → Request Validator . . . 9
- 1.14 Read Path: Request Validator → Tag Allocator / Error Handler . . . 9
- 1.15 Read Path: Error Handler → Tag Allocator . . . 10
- 1.16 Shared: Burst Splitter → Packet Formation Unit (Read) . . . 10
- 1.17 Read Path: Packet Formation Unit → Read Packet FIFO . . . 10
- 1.18 Read Path: Read Packet FIFO → MC Core . . . 11
- 1.19 MC Core Completion Interface (MC Core → CIF) . . . 11
- 1.20 Read Path: ROB → Merge Logic . . . 12
- 1.21 Read Path: RTT → ROB / Merge Logic . . . 12
- 1.22 Read Path: Merge Logic → R Response Generator . . . 13
- 1.23 ROB Backpressure Control . . . 13

2 Arbitration Policy **15**
- 2.1 Overview . . . 15
- 2.2 Transaction Tag Allocator . . . 15
- 2.3 Burst Splitter . . . 16

3 Reset and Initialization Sequence **17**
- 3.1 Overview . . . 17
- 3.2 Reset Signal . . . 17
- 3.3 Power-On Reset Sequence . . . 17
- 3.4 Soft Reset Sequence . . . 18
- 3.5 BRAM Initialization Note . . . 18

4 Error Propagation Timing **19**

4.1 Overview 19
4.2 Write Path Error Propagation 20
4.3 Read Path Error Propagation 21
4.4 Split Transaction Error Propagation 21
4.5 Err_write W Beat Handling 22
4.6 Timing Summary 22

1 Interface Signal Table

This section enumerates every signal crossing a block boundary within the CIF. Signals are grouped by boundary in pipeline order: AXI slave interface first, then write path, then read path, then shared blocks, then the CIF–MC Core interface.

Handshake types used below:

- **VALID/READY** — standard AXI4 handshake; transfer occurs when both asserted.
- **VALID/READY (tied)** — READY permanently high; transfer occurs on VALID.
- **Pulse** — single-cycle strobe, no handshake.
- **Level** — combinational or registered level signal, no handshake.
- **FIFO push/pop** — implicit handshake via FIFO full/empty flags.

1.1 AXI Slave Interface (AXI Master → CIF)

Signal	W	Dir	From	To	Handshake	Description
AWID[5:0]	6	in	AXI Master	AW Meta FIFO	VALID/READY	Write transaction ID
AWADDR[31:0]	32	in	AXI Master	AW Meta FIFO	VALID/READY	Write address
AWLEN[7:0]	8	in	AXI Master	AW Meta FIFO	VALID/READY	Burst length minus 1
AWSIZE[2:0]	3	in	AXI Master	AW Meta FIFO	VALID/READY	Transfer size
AWBURST[1:0]	2	in	AXI Master	AW Meta FIFO	VALID/READY	Burst type (INCR/WRAP)
AWVALID	1	in	AXI Master	AW Meta FIFO	VALID/READY	AW channel valid
AWREADY	1	out	AW Meta FIFO	AXI Master	VALID/READY	AW channel ready; deasserted when client slot full
WDATA[511:0]	512	in	AXI Master	W Data FIFO	VALID/READY	Write data
WSTRB[63:0]	64	in	AXI Master	W Data FIFO	VALID/READY	Write byte strobes
WLAST	1	in	AXI Master	W Data FIFO	VALID/READY	Last beat of burst
WVALID	1	in	AXI Master	W Data FIFO	VALID/READY	W channel valid

WREADY	1	out	W Data FIFO	AXI Master	VALID/READY	W channel ready; deasserted when FIFO full
BVALID	1	out	B Resp Gen	AXI Master	VALID/READY	B channel valid
BREADY	1	in	AXI Master	B Resp Gen	VALID/READY	B channel ready
BRESP[1:0]	2	out	B Resp Gen	AXI Master	VALID/READY	Write response (OKAY/DECERR)
BID[5:0]	6	out	B Resp Gen	AXI Master	VALID/READY	Write response ID
ARID[5:0]	6	in	AXI Master	AR Meta FIFO	VALID/READY	Read transaction ID
ARADDR[31:0]	32	in	AXI Master	AR Meta FIFO	VALID/READY	Read address
ARLEN[7:0]	8	in	AXI Master	AR Meta FIFO	VALID/READY	Burst length minus 1
ARSIZE[2:0]	3	in	AXI Master	AR Meta FIFO	VALID/READY	Transfer size
ARBURST[1:0]	2	in	AXI Master	AR Meta FIFO	VALID/READY	Burst type (INCR/WRAP)
ARVALID	1	in	AXI Master	AR Meta FIFO	VALID/READY	AR channel valid
ARREADY	1	out	AR Meta FIFO	AXI Master	VALID/READY	AR channel ready; deasserted when client slot full
RVALID	1	out	R Resp Gen	AXI Master	VALID/READY	R channel valid
RREADY	1	in	AXI Master	R Resp Gen	VALID/READY	R channel ready
RDATA[511:0]	512	out	R Resp Gen	AXI Master	VALID/READY	Read data
RRESP[1:0]	2	out	R Resp Gen	AXI Master	VALID/READY	Read response (OKAY/DECERR)
RID[5:0]	6	out	R Resp Gen	AXI Master	VALID/READY	Read response ID
RLAST	1	out	R Resp Gen	AXI Master	VALID/READY	Last beat of read burst

1.2 Write Path: AW Metadata FIFO → Request Validator

Signal	W	Dir	From	To	Handshake	Description
AWADDR[31:0]	32	out	AW Meta FIFO	Req Validator	FIFO pop	Write address from FIFO head
AWID[5:0]	6	out	AW Meta FIFO	Req Validator	FIFO pop	AXI ID from FIFO head
AWLEN[7:0]	8	out	AW Meta FIFO	Req Validator	FIFO pop	Burst length from FIFO head
AWSIZE[2:0]	3	out	AW Meta FIFO	Req Validator	FIFO pop	Transfer size from FIFO head
AWBURST[1:0]	2	out	AW Meta FIFO	Req Validator	FIFO pop	Burst type from FIFO head
STALL	1	in	Tag Allocator	Req Validator	Level	Per-AXID stall; Req Validator holds pop when asserted

1.3 Write Path: Request Validator → Tag Allocator / Error Handler

Signal	W	Dir	From	To	Handshake	Description
REQ_VALID	1	out	Req Validator	Tag Allocator	Pulse	Valid transaction; address check passed
AXID[5:0]	6	out	Req Validator	Tag Allocator	Pulse	AXI ID of valid request
AWLEN[7:0]	8	out	Req Validator	Tag Allocator	Pulse	Burst length of valid request
ERR_VALID	1	out	Req Validator	Error Handler	Pulse	Address error notification

AXID[5:0]	6	out	Req Validator	Error Handler	Pulse	AXI ID preserved for NOP tag allocation

1.4 Write Path: Error Handler → Tag Allocator

Signal	W	Dir	From	To	Handshake	Description
REQ_VALID	1	out	Error Handler	Tag Allocator	Pulse	NOP allocation request; pkt_type pre-set to Err_write
AXID[5:0]	6	out	Error Handler	Tag Allocator	Pulse	AXI ID for NOP tag allocation

1.5 Shared: Tag Allocator → Burst Splitter

Signal	W	Dir	From	To	Handshake	Description
TXN_TAG[7:0]	8	out	Tag Allocator	Burst Splitter	Pulse	Allocated tag; [7:2]=AXID, [1:0]=seqnum
TAG_VALID	1	out	Tag Allocator	Burst Splitter	Pulse	Tag output valid; registered, valid cycle after alloc_en
STALL	1	out	Tag Allocator	Req Validator / Error Handler	Level	Per-AXID stall; asserted when occupancy[AXID]==4

1.6 Shared: Tag Allocator → Response Tracker / RTT

Signal	W	Dir	From	To	Handshake	Description
TXN_TAG[7:0]	8	out	Tag Allocator	Resp Tracker	Pulse	Allocated write tag; indexes Response Tracker register file

AWLEN[7:0]	8	out	Tag Allocator	Resp Tracker	Pulse	Write burst length; exp_pkts = AWLEN+1
TAG_VALID	1	out	Tag Allocator	Resp Tracker	Pulse	Allocation strobe to Response Tracker
TXN_TAG[7:0]	8	out	Tag Allocator	RTT	Pulse	Allocated read tag; indexes RTT register file
ARLEN[7:0]	8	out	Tag Allocator	RTT	Pulse	Read burst length; exp_pkts = ARLEN+1
TAG_VALID	1	out	Tag Allocator	RTT	Pulse	Allocation strobe to RTT
alloc_req	1	out	Tag Allocator	ROB	Pulse	ROB slot allocation request, coincident with read TAG_VALID
AXID[5:0]	6	out	Tag Allocator	ROB	Pulse	AXI ID for ROB slot allocation
seqnum[1:0]	2	out	Tag Allocator	ROB	Pulse	Seqnum for ROB lookup table write

1.7 Shared: Burst Splitter → Packet Formation Unit (Write)

Signal	W	Dir	From	To	Handshake	Description
pkt_valid	1	out	Burst Splitter	Write PFU	VALID/READY	Normalized beat context valid
pkt_ready	1	in	Write PFU	Burst Splitter	VALID/READY	Downstream ready; holds Emit FSM when low
addr[31:0]	32	out	Burst Splitter	Write PFU	VALID/READY	Per-beat write address

TXN_TAG[7:0]	8	out	Burst Splitter	Write PFU	VALID/READY	Transaction tag
pkt_type[1:0]	2	out	Burst Splitter	Write PFU	VALID/READY	00=Write, 10=Err_write; carried unchanged from Stage 1
last_beat	1	out	Burst Splitter	Write PFU	VALID/READY	Final beat of sub-transaction
s1_busy	1	out	Burst Splitter	AW Meta FIFO	Level	Stalls Stage 1 FIFO pop during EMIT B state

1.8 Write Path: W Data FIFO → Packet Formation Unit

Signal	W	Dir	From	To	Handshake	Description
WDATA[511:0]	512	out	W Data FIFO	Write PFU	FIFO pop	Write data beat
WSTRB[63:0]	64	out	W Data FIFO	Write PFU	FIFO pop	Byte enable strobes
WLAST	1	out	W Data FIFO	Write PFU	FIFO pop	Last beat flag

1.9 Write Path: Packet Formation Unit → Write Packet FIFO

Signal	W	Dir	From	To	Handshake	Description
PKT_DATA[617:0]	618	out	Write PFU	Write Pkt FIFO	FIFO push	Serialized 618-bit write packet: {wrapper, data, addr, txn_tag, pkt_type}
PKT_VALID	1	out	Write PFU	Write Pkt FIFO	FIFO push	Packet push strobe

1.10 Write Path: Write Packet FIFO → MC Core

Signal	W	Dir	From	To	Handshake	Description

PKT_DATA[617:0]	618	out	Write Pkt FIFO	MC Core	VALID/READY	Serialized write packet
PKT_VALID	1	out	Write Pkt FIFO	MC Core	VALID/READY	Packet valid
PKT_READY	1	in	MC Core	Write Pkt FIFO	VALID/READY	MC Core ready to accept; back-pressure propagates upstream

1.11 Write Path: Response Tracker → B Response Generator

Signal	W	Dir	From	To	Handshake	Description
BRESP_VALID	1	out	Resp Tracker	B Resp Gen	Pulse	BRESP ready to emit; asserted cycle after txn_done evaluated
BID[5:0]	6	out	Resp Tracker	B Resp Gen	Pulse	AXI ID recovered from head_tag[7:2]
BRESP[1:0]	2	out	Resp Tracker	B Resp Gen	Pulse	OKAY if err_sticky==0; DECERR if err_sticky==1

1.12 Write Path: Response Tracker → Tag Allocator (Free)

Signal	W	Dir	From	To	Handshake	Description
WR_FREE_VALID	1	out	Resp Tracker	Tag Allocator	Pulse	Write slot free strobe; coincident with BRESP emission

WR_FREE_TAG[7:0]	8	out	Resp Tracker	Tag Allocator	Pulse	Tag being freed; [7:2]=AXID for occupancy decrement

1.13 Read Path: AR Metadata FIFO → Request Validator

Signal	W	Dir	From	To	Handshake	Description
ARADDR[31:0]	32	out	AR Meta FIFO	Req Validator	FIFO pop	Read address from FIFO head
ARID[5:0]	6	out	AR Meta FIFO	Req Validator	FIFO pop	AXI ID from FIFO head
ARLEN[7:0]	8	out	AR Meta FIFO	Req Validator	FIFO pop	Burst length from FIFO head
ARSIZE[2:0]	3	out	AR Meta FIFO	Req Validator	FIFO pop	Transfer size from FIFO head
ARBURST[1:0]	2	out	AR Meta FIFO	Req Validator	FIFO pop	Burst type from FIFO head
STALL	1	in	Tag Allocator	Req Validator	Level	Per-AXID stall; Req Validator holds pop when asserted

1.14 Read Path: Request Validator → Tag Allocator / Error Handler

Signal	W	Dir	From	To	Handshake	Description
REQ_VALID	1	out	Req Validator	Tag Allocator	Pulse	Valid read request; address check passed
AXID[5:0]	6	out	Req Validator	Tag Allocator	Pulse	AXI ID of valid read request
ARLEN[7:0]	8	out	Req Validator	Tag Allocator	Pulse	Burst length of valid read request

ERR_VALID	1	out	Req Validator	Error Handler	Pulse	Address error notification
AXID[5:0]	6	out	Req Validator	Error Handler	Pulse	AXI ID preserved for NOP tag allocation

1.15 Read Path: Error Handler → Tag Allocator

Signal	W	Dir	From	To	Handshake	Description
REQ_VALID	1	out	Error Handler	Tag Allocator	Pulse	NOP allocation request; pkt_type pre-set to Err_read
AXID[5:0]	6	out	Error Handler	Tag Allocator	Pulse	AXI ID for NOP tag allocation

1.16 Shared: Burst Splitter → Packet Formation Unit (Read)

Signal	W	Dir	From	To	Handshake	Description
pkt_valid	1	out	Burst Splitter	Read PFU	VALID/READY	Normalized beat context valid
pkt_ready	1	in	Read PFU	Burst Splitter	VALID/READY	Downstream ready
addr[31:0]	32	out	Burst Splitter	Read PFU	VALID/READY	Per-beat read address
TXN_TAG[7:0]	8	out	Burst Splitter	Read PFU	VALID/READY	Transaction tag
pkt_type[1:0]	2	out	Burst Splitter	Read PFU	VALID/READY	01=Read, 11=Err_read
last_beat	1	out	Burst Splitter	Read PFU	VALID/READY	Final beat of sub-transaction
s1_busy	1	out	Burst Splitter	AR Meta FIFO	Level	Stalls Stage 1 FIFO pop during EMIT B state

1.17 Read Path: Packet Formation Unit → Read Packet FIFO

Signal	W	Dir	From	To	Handshake	Description
PKT_DATA[41:0]	42	out	Read PFU	Read Pkt FIFO	FIFO push	Serialized 42-bit read packet: {addr, txn_tag, pkt_type}
PKT_VALID	1	out	Read PFU	Read Pkt FIFO	FIFO push	Packet push strobe

1.18 Read Path: Read Packet FIFO → MC Core

Signal	W	Dir	From	To	Handshake	Description
PKT_DATA[41:0]	42	out	Read Pkt FIFO	MC Core	VALID/READY	Serialized read request packet
PKT_VALID	1	out	Read Pkt FIFO	MC Core	VALID/READY	Packet valid
PKT_READY	1	in	MC Core	Read Pkt FIFO	VALID/READY	MC Core ready; deasserted by ROB watermark

1.19 MC Core Completion Interface (MC Core → CIF)

Signal	W	Dir	From	To	Handshake	Description
RESP_VALID	1	in	MC Core	ROB / Resp Tracker / RTT	VALID/READY (tied)	Completion valid
RESP_READY	1	out	CIF	MC Core	VALID/READY (tied)	Tied permanently high; CIF never stalls MC Core completions
resp_type[1:0]	2	in	MC Core	ROB / Resp Tracker / RTT	VALID/READY (tied)	00=RD_DATA (→ROB, RTT), 01=WR_ACK (→Resp Tracker), 10=ERR

tag[7:0]	8	in	MC Core	ROB / Resp Tracker / RTT	VALID/READY (tied)	Completion tag; [7:2]=AXID, [1:0]=seqnum
data[511:0]	512	in	MC Core	ROB	VALID/READY (tied)	Read data; valid only when resp_type==00
status[3:0]	4	in	MC Core	ROB / Resp Tracker	VALID/READY (tied)	Status/error code; encoding TBD with MC team

1.20 Read Path: ROB → Merge Logic

Signal	W	Dir	From	To	Handshake	Description
data[511:0]	512	out	ROB	Merge Logic	VALID/READY	In-order read data beat emitted by Emit Logic
status[3:0]	4	out	ROB	Merge Logic	VALID/READY	Status for emitted beat
AXID[5:0]	6	out	ROB	Merge Logic	VALID/READY	AXID of emitted beat
rob_last	1	out	RTT (via ROB)	Merge Logic	VALID/READY	Final beat of transaction; sourced from RTT, passed through ROB coincident with final data beat

1.21 Read Path: RTT → ROB / Merge Logic

Signal	W	Dir	From	To	Handshake	Description

rob_last	1	out	RTT	ROB / Merge Logic	Pulse	One-cycle pulse on final MC Core completion for a transaction; cmp_pkts == exp_pkts
RD_FREE_VALID	1	out	RTT	Tag Allocator	Pulse	Read slot free strobe; coincident with rob_last
RD_FREE_TAG[7:0]	8	out	RTT	Tag Allocator	Pulse	Tag being freed; [7:2]=AXID for occupancy decrement

1.22 Read Path: Merge Logic → R Response Generator

Signal	W	Dir	From	To	Handshake	Description
RDATA[511:0]	512	out	Merge Logic	R Resp Gen	VALID/READY	Ordered read data beat
RID[5:0]	6	out	Merge Logic	R Resp Gen	VALID/READY	AXI ID recovered from txn_tag[7:2]
status[3:0]	4	out	Merge Logic	R Resp Gen	VALID/READY	Beat status; drives DECERR when error
RLAST	1	out	Merge Logic	R Resp Gen	VALID/READY	Last beat flag derived from rob_last

1.23 ROB Backpressure Control

Signal	W	Dir	From	To	Handshake	Description

watermark_hit	1	out	ROB Occ Counter	PKT_READY ctrl	Level	Asserted when occupancy $\geq$ 38; combinational from counter
PKT_READY	1	out	ROB	Read Pkt FIFO	Level	Deasserted when watermark_hit high; halts new read requests to MC Core

2 Arbitration Policy

2.1 Overview

Two blocks in CIF are shared across all $N\_CLIENTS = 32$ AXI clients: the **Transaction Tag Allocator** and the **Burst Splitter**. Both sit on the common request path after the per-path Request Validators. This section defines how access to these shared resources is arbitrated.

No other block requires explicit arbitration. The AR and AW Metadata FIFOs are partitioned statically (4 slots per client); the ROB, RTT, and Response Tracker are indexed directly by `txn_tag` and require no arbitration. The completion path from MC Core is a single bus fanned out to the ROB, RTT, and Response Tracker simultaneously, with each block filtering on `resp_type` — no arbiter required.

2.2 Transaction Tag Allocator

The Tag Allocator is a single shared instance serving both read and write paths.

Arbitration Mechanism

The Tag Allocator accepts exactly one allocation request per cycle. Upstream priority is resolved as follows:

1. **Write path** and **read path** requests arrive at the Tag Allocator from their respective Request Validators (or Error Handlers). In CIF v1, write and read paths share the Burst Splitter sequentially (single pipeline); the upstream MUX feeding the Tag Allocator selects between write-path and read-path requests using **fixed priority, write > read**. If both paths present a request in the same cycle, the write-path request is accepted; the read-path request is held for the next cycle.
2. Within a single path, only one AXID is presented per cycle (the FIFO head entry). No further arbitration is required at this level.

Per-AXID Stall

Allocation for a given AXID is blocked when `occupancy[AXID] == MAX_OUTSTANDING` (4). The Stall Logic asserts `STALL` combinationally to the upstream block for that specific AXID only. Other AXIDs are unaffected. `STALL` is released in the cycle after the occupancy counter decrements (i.e., cycle $N + 1$ after `WR_FREE_VALID` or `RD_FREE_VALID` fires for a tag belonging to that AXID).

Error Path

The Error Handler presents allocation requests using the same interface as the Request Validator (`REQ_VALID`, `AXID`). Error-path allocations are subject to the same per-AXID stall and write/read priority rules. No bypass or priority elevation is granted to error transactions.

Simultaneous Free and Allocate

If `alloc_en` and `dec_occ` fire in the same cycle for the same AXID (one slot freed while a new one is being allocated), the occupancy counter holds constant (net-zero change). This is a valid steady-state operating condition and requires no special handling.

2.3 Burst Splitter

The Burst Splitter is a single shared two-stage pipeline instance serving both read and write paths across all clients.

Arbitration Mechanism

Access to the Burst Splitter is implicitly arbitrated by the upstream MUX that feeds the Tag Allocator. Since the Tag Allocator and Burst Splitter are driven from the same upstream MUX with the same priority policy (write > read, one request per cycle), the Burst Splitter processes exactly one transaction at a time.

Pipeline Stall Policy

The Burst Splitter Emit FSM stalls Stage 1 (via `s1_busy`) during EMIT B to prevent the next FIFO entry from being consumed while the second sub-transaction of a split is being emitted. `s1_busy` is deasserted on the cycle the FSM returns to IDLE. Backpressure from downstream (`pkt_ready = 0`) holds the FSM in its current state; `s1_busy` remains asserted until IDLE is reached.

Tag Allocator Stall Folding

If the Tag Allocator asserts `STALL` before Stage 1 can proceed, the stall is folded into `s1_busy`. Stage 1 does not advance until `STALL` deasserts and a valid tag is available.

Throughput

Under no-split, no-stall conditions the Burst Splitter achieves one transaction per cycle. A row-boundary split costs one additional cycle per transaction (EMIT A + EMIT B). WRAP bursts iterate over the pre-computed `beat_addrs[]` table; throughput is one beat per cycle per WRAP address, plus one additional cycle if a row-boundary split is also required.

3 Reset and Initialization Sequence

3.1 Overview

CIF supports two reset modes:

- **Power-on reset (POR):** Asserted by the FPGA fabric on configuration. All flip-flops and BRAM init values take effect. CIF remains in reset until the AXI interconnect and MC Core are both ready.
- **Soft reset:** Driven by a register write from a management plane. Must drain the pipeline before asserting; returns CIF to the same state as POR without reconfiguring the FPGA.

All resets are **synchronous, active-high**, qualified on `posedge clk`. There is no asynchronous reset path in CIF.

3.2 Reset Signal

Signal	W	Source	Description
`rst_n`	1	FPGA top / mgmt plane	Active-low synchronous reset; asserted low to reset all CIF blocks

3.3 Power-On Reset Sequence

The following sequence describes the required initialization order. Steps within the same numbered stage may proceed in parallel.

1. **Assert `rst_n = 0`.** All registered state is cleared synchronously on the next `posedge clk`:
   - AR/AW Metadata FIFO and W Data FIFO: empty, `ARREADY / AWREADY / WREADY = 0`.
   - Write and Read Packet FIFOs: empty.
   - Tag Allocator: all 64 occupancy counters ← 0; all 64 seqnum counters ← 0.
   - Response Tracker register file: all entries cleared (`cmp_pkts = 0`, `err_sticky = 0`, `exp_pkts = 0`).
   - Response Tracker head pointer array: all 64 entries ← 0.
   - RTT register file: all 256 entries cleared.
   - ROB BRAM: all 64 slots `valid = 0`.
   - ROB free list: all 64 bits ← 1 (all slots free).
   - ROB lookup table: all 256 entries ← 0.
   - ROB head pointer array: all 64 registers ← 0.
   - ROB occupancy counter ← 0.
   - Burst Splitter Emit FSM ← IDLE; all internal state registers cleared.
   - Request Validators: combinational; no state to clear.
   - Error Handlers: no persistent state to clear.
   - `RESP_READY` driven high immediately (tied; no reset dependency).

2. **Wait for MC Core ready.** CIF must not issue any packets until MC Core signals it is ready to accept traffic. `PKT_VALID` must be held low (Packet FIFOs are empty during reset; no additional gating required if FIFOs reset correctly).
3. **Deassert `rst_n = 1`.** On the first `posedge clk` after deassertion:
   - `AWREADY` and `ARREADY` are driven based on per-client FIFO slot availability. Both are high immediately after reset since all FIFO slots are free.
   - `WREADY` is driven high immediately (W Data FIFO empty).
   - The Tag Allocator begins accepting allocation requests.
   - The Burst Splitter Emit FSM begins processing from IDLE.
4. **AXI traffic may begin.** First valid AXI transaction is accepted on the cycle after `AWREADY / ARREADY` are high and the AXI master presents a valid request.

3.4 Soft Reset Sequence

Soft reset must be preceded by a pipeline drain to avoid losing in-flight transactions.

1. **Quiesce upstream.** The management plane instructs all AXI masters to stop issuing new transactions. New `AWVALID` and `ARVALID` are withheld.
2. **Drain the request pipeline.** Wait until:
   - AR and AW Metadata FIFOs are empty.
   - W Data FIFO is empty.
   - Write and Read Packet FIFOs are empty.
   - Burst Splitter Emit FSM is in IDLE.
   - Tag Allocator: all 64 occupancy counters = 0 (no in-flight transactions).
3. **Drain the completion pipeline.** Wait until:
   - ROB occupancy counter = 0 (all read completions have been emitted to Merge Logic).
   - Response Tracker: all 256 register file entries clear (`cmp_pkts == exp_pkts` and freed, or all `exp_pkts == 0`).
   - RTT: all 256 entries clear.
   - No pending `BRESP` or `RRESP` outstanding on the AXI B or R channels.
4. **Assert `rst_n = 0`.** Proceed identically to steps 1–4 of the power-on reset sequence.

3.5 BRAM Initialization Note

The ROB BRAM `valid` bits are the only BRAM state that must be explicitly initialized. On the cycle `rst_n` deasserts, all 64 `valid` bits must read as 0. This is achieved by:

- Using a synchronous-reset flip-flop for each `valid` bit (implemented as distributed RAM or shadow registers alongside the BRAM data), **or**
- Performing a 64-cycle initialization write loop on the BRAM write port during reset, setting `valid = 0` at each address before `rst_n` deasserts.

The second option requires holding `rst_n` low for at least 64 cycles after POR. This is the recommended approach for UltraScale BRAM inference.

4 Error Propagation Timing

4.1 Overview

Address errors detected by the Request Validator are propagated through the normal CIF pipeline as NOP packets using `pkt_type[1:0] = 10` (`Err_write`) or `11` (`Err_read`). This section traces the cycle-by-cycle timing of `pkt_type` from error detection to AXI error response, and identifies where `pkt_type` is set, carried, and consumed. Timing counts assume no stalls unless noted.

4.2 Write Path Error Propagation

Cycle	Stage	Event
N	Request Validator	Address Range Check evaluates `addr_ok = 0` combinationally. `ERR_VALID` asserted to Error Handler; `AXID` forwarded. AW Metadata FIFO entry is consumed (popped) in this cycle.
N	Error Handler	Error Handler sets `pkt_type = Err_write` and presents `REQ_VALID` + `AXID` to Tag Allocator in the same cycle.
$N+1$	Tag Allocator	`alloc_en` fires (assuming no `STALL`). `TXN_TAG` and `TAG_VALID` registered; valid on cycle $N+1$. Occupancy counter and seqnum counter update take effect on cycle $N+1$.
$N+1$	Burst Splitter Stage 1	`pkt_type = Err_write` enters Stage 1 alongside `TXN_TAG`, `ADDR`, `AXLEN`, `AXSIZE`. Stage 1 computes split decision combinationally; pipeline register clocked on cycle $N+1$.
$N+2$	Burst Splitter Stage 2	Emit FSM enters EMIT A. `pkt_type` carried unchanged through Sub-txn A mux. Both sub-transactions (if split) carry the same `pkt_type = Err_write`.
$N+2$	Packet Formation Unit	Write PFU assembles 618-bit packet with `pkt_type[1:0] = 10` in bits [1:0]. `data[511:0]` and `wrapper[63:0]` are taken from the W Data FIFO as-is; no drain or discard.
$N+2$	Write Packet FIFO	Packet pushed into FIFO.
$N+2+F_W$	MC Core	`Err_write` packet dequeued from Write Packet FIFO and issued to MC Core (`PKT_VALID` asserted). F_W = Write Packet FIFO occupancy delay (0 under no backpressure). MC Core discards write data; returns error completion (`resp_type = 10`) on the completion bus.
$N+2+F_W+L_{MC}$	Response Tracker	MC Core asserts `RESP_VALID` with `resp_type = 10`. Completion Update Logic increments `cmp_pkts` and sets `err_sticky = 1`. Compare Logic evaluates `cmp_pkts == exp_pkts` combinationally (true on this cycle for a single-packet error transaction). `txn_done` pulses.
$N+3+F_W+L_{MC}$	B Response Generator	Emit Logic (registered) drives `BRESP_VALID`, `BID`, and `BRESP = DECERR` to B Response Generator on the cycle after `txn_done`. `BVALID` is asserted on the AXI B channel.

L_{MC} denotes MC Core's internal latency from packet receipt to error completion assertion. This is a MC Core parameter and is not constrained by CIF.

Key invariant: `pkt_type` is set once (at the Error Handler output, cycle N) and carried unchanged through the S1→S2 pipeline register and Sub-txn muxes. Neither the Burst Splitter nor the PFU inspect or modify `pkt_type`. Both sub-transactions of a split carry the same `pkt_type`.

4.3 Read Path Error Propagation

Cycle	Stage	Event
N	Request Validator	`addr_ok = 0` combinationally. `ERR_VALID` asserted to Error Handler; `AXID` forwarded. AR Metadata FIFO entry consumed.
N	Error Handler	`pkt_type = Err_read` set. `REQ_VALID` + `AXID` presented to Tag Allocator.
$N+1$	Tag Allocator	`TXN_TAG` and `TAG_VALID` registered and valid. ROB slot allocated via `alloc_req`. RTT entry allocated via `TAG_VALID` path with `exp_pkts = ARLEN+1`.
$N+1$	Burst Splitter Stage 1	`pkt_type = Err_read` enters Stage 1; pipeline register clocked.
$N+2$	Burst Splitter Stage 2	Emit FSM emits beat(s) with `pkt_type = Err_read` unchanged.
$N+2$	Packet Formation Unit	Read PFU assembles 42-bit packet with `pkt_type[1:0] = 11` in bits [1:0]. No data field in read packets.
$N+2$	Read Packet FIFO	Packet pushed into FIFO.
$N+2+F_R$	MC Core	`Err_read` packet issued to MC Core. MC Core returns error completion (`resp_type = 10`) with no data.
$N+2+F_R+L_{MC}$	ROB	Completion Input writes `status` and `valid = 1` to the allocated ROB slot with error status preserved. RTT: `cmp_pkts` incremented; `rob_last` pulses (single-beat transaction).
$N+2+F_R+L_{MC}$	Merge Logic	Emit Logic emits slot to Merge Logic with `status` indicating error. `rob_last` forwarded simultaneously.
$N+3+F_R+L_{MC}$	R Response Generator	`RVALID` asserted with `RRESP = DECERR`, `RID` recovered from `tag[7:2]`, `RLAST = 1`.

4.4 Split Transaction Error Propagation

When an error transaction also requires a row-boundary split (`split_needed = 1`), both sub-transactions carry the same `pkt_type`. The implications are:

- **Write path:** Both `Err_write` packets are issued to MC Core. MC Core returns one error completion per packet. The Response Tracker increments `cmp_pkts` on each completion; `err_sticky` is set on the first. `txn_done` fires only after both completions arrive (`cmp_pkts == exp_pkts`). `BRESP = DECERR` is emitted once, covering the entire transaction.
- **Read path:** Both `Err_read` packets are issued to MC Core. MC Core returns one error completion per packet (no data). The RTT increments `cmp_pkts` on each; `rob_last` fires after the second. The ROB emits two slots (both with error status); Merge Logic forwards both to the R Response Generator. `RLAST` is asserted on the final beat; both beats carry `RRESP = DECERR`.
- **Cycle cost of split:** EMIT B adds one cycle to the Burst Splitter stage (Stage 2 holds `s1_busy` for one additional cycle). All downstream stages (PFU, FIFO, MC Core, response path) are otherwise unaffected by the split.

4.5 Err_write W Beat Handling

W beats for an `Err_write` transaction flow through the W Data FIFO and Write PFU without modification or draining. The `pkt_type` field in the assembled packet signals to MC Core that the data is invalid; MC Core discards the payload. No special W-beat gating or bypass path exists in CIF. The number of W beats consumed by the PFU is determined by `AWLEN`, consistent with the normal write path.

4.6 Timing Summary

Stage	**Write Error Latency**	**Read Error Latency**
Req Validator → Error Handler	0 (combinational)	0 (combinational)
Error Handler → Tag Allocator output	1 cycle	1 cycle
Tag Allocator → Burst Splitter S2 output	1 cycle	1 cycle
Burst Splitter S2 → Packet FIFO push	0 (same cycle)	0 (same cycle)
Packet FIFO → MC Core (no backpressure)	0	0
MC Core latency	L_{MC}	L_{MC}
Final completion → BRESP/RRESP (no split)	1 cycle (Emit Logic reg.)	0 (ROB emit + Merge Log
Total CIF-internal latency (no split, no stall)	$\mathbf{3 + L_{MC}}$ **cycles**	$\mathbf{2 + L_{MC}}$ **cycles**